

COMSATS University Islamabad, Lahore Campus

Theory Assignment 1

Submitted By: SYED TURAB HAIDER

Registration Number: SP26-BCS-050

Submitted To: Dr. Muhammad Shahid Bhatti

TASK 1

1. Stakeholders and Their Needs

The Smart Campus Learning Space Allocation System has several stakeholders with different requirements.

Stakeholder	Need
Students	Need fair access to classrooms, meeting rooms and project spaces. They should be able to see available spaces without seeing private reservation details.
Faculty Members	Need reliable rooms for scheduled teaching and urgent academic activities.
Administrators	Need to manage rooms, reservations, approvals and room utilization.
Security Staff	Need traceable access records for security and auditing purposes.
Maintenance Staff	Need to immediately block rooms that are unsafe or require maintenance.
Disability Support Staff	Need accessible rooms to remain available for users who require accessibility features.

The scenario identifies students, faculty, administrators, security staff, maintenance staff and disability support staff as stakeholders.

2. Requirement Conflicts

Conflict 1: Fair Access vs Faculty Priority

Students expect fair access to spaces, while faculty may require priority for scheduled teaching and urgent academic activities.

Decision: Scheduled teaching will receive priority over ordinary requests. When requests have the same priority, student requests will be handled using a first-come-first-served approach.

Conflict 2: Privacy vs Security Traceability

Security staff require traceable records, but ordinary users should not see the identity or purpose of another person's reservation.

Decision: Authorized security and administrative staff can access necessary records for auditing. Ordinary users will only see whether a room is available or unavailable.

This follows the Privacy First condition for registration numbers ending in 0 or 1.

Conflict 3: Room Utilization vs Safety

Administration wants high room utilization, but maintenance staff must be able to block unsafe rooms immediately.

Decision: Safety will take priority over utilization. Once a room is marked unsafe, new reservations will not be allowed for it.

3. Facts and Assumptions

Facts given in the scenario: The system manages classrooms, laboratories, seminar rooms and project spaces. Different activities have different capacity, equipment and approval requirements. The scenario also requires fairness, priority handling, privacy, traceability and emergency handling.

My assumptions: Scheduled teaching has priority over ordinary requests; equal-priority requests are handled fairly; ordinary users only see availability; authorized staff can access audit information; and safety blocks have priority over utilization.

TASK 2

The proposed system contains nine main classes.

Class	Main Responsibility	Attributes	Operations
User	Represents a person using the system	userID, name, email	requestReservation(), cancelReservation()
Role	Represents a user's role and permissions	roleID, roleName, permissions	addPermission(), removePermission()
Room	Manages learning-space information and availability	roomID, capacity, status	checkAvailability(), blockRoom()

Class	Main Responsibility	Attributes	Operations
Reservation	Represents a request/reservation for a room	reservationID, startTime, endTime, status	confirm(), cancel(), relocate()
Activity	Describes the purpose and requirements of a reservation	activityType, requiredCapacity, requiredEquipment	checkRequirements()
Approval	Represents an approval decision	approvalID, decision, decisionTime	approve(), reject()
Equipment	Represents equipment available in a room	equipmentID, equipmentType, availability	reserveEquipment(), releaseEquipment()
MaintenanceBlock	Represents a period during which a room cannot be used	blockID, reason, startTime, endTime	activateBlock(), endBlock()
Notification	Sends information about reservation changes	notificationID, message, timestamp	send(), markAsRead()

Main UML Relationships

User 1 ----- 0..* Reservation

User 1 ----- 1..* Role

Reservation 1 ----- 1 Room

Reservation 1 ----- 1 Activity

Reservation 1 ----- 0..1 Approval

Room 1 ----- 0..* Equipment

Room 1 ----- 0..* MaintenanceBlock

Reservation 1 ----- 0..* Notification

User 1 ----- 0..* Notification

Relationship Explanation

A User can make many reservations and can have multiple roles. A Reservation is associated with exactly one room and one activity. A room can contain multiple equipment items and can have multiple maintenance blocks over time.

Composition is appropriate where an object strongly owns its parts and their lifecycle. Aggregation/association is more appropriate where objects can exist independently. For example, a reservation is associated with a room rather than owning the room itself.

Responsibility Separation

The design avoids a God Class. Each class has a focused responsibility. For example, Room manages room availability, Reservation manages reservation states, and Role manages permissions. This makes the conceptual model easier to understand and maintain.

TASK 3

1. Abstraction

User

The **User** class abstracts a real campus person who interacts with the system. It does not need to represent every personal detail of the person. It mainly represents information needed for reservations and permissions.

Room

The **Room** class abstracts a physical campus space. Important concepts such as capacity, availability and room status are represented without modelling every physical detail of the room.

Reservation

The **Reservation** class abstracts the real-world process of requesting and booking a campus space. It represents information such as time, room, activity and reservation status.

2. Encapsulation

The internal data of important objects should not be directly changed by any user.

For example:

- **Room.status** should be changed through controlled operations such as `blockRoom()` rather than directly.
- **Reservation.status** should be changed through operations such as `confirm()` and `cancel()`.
- **User permissions** should be controlled through the **Role** object and `hasPermission()`.

This prevents unauthorized or invalid changes.

For the **Privacy First** requirement, reservation identity and purpose should also be protected. Ordinary users should use an operation such as `viewAvailability()` that returns availability information without exposing private reservation details.

3. Invariants

An invariant is a rule that must always remain true.

Invariant 1: A confirmed reservation must have one valid room and a valid time period.

Invariant 2: Two confirmed reservations cannot occupy the same room at overlapping times.

Invariant 3: A room marked unsafe or blocked by maintenance cannot receive a new confirmed reservation.

Invariant 4: Ordinary users cannot access the identity or purpose of another user's reservation.

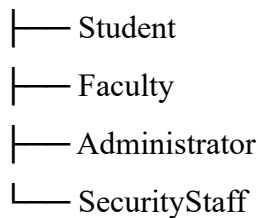
Invariant 5: A reservation's end time must always be later than its start time.

Encapsulation helps maintain these invariants because changes are made through controlled operations rather than unrestricted direct access to object data.

TASK 4

The system could represent roles using inheritance:

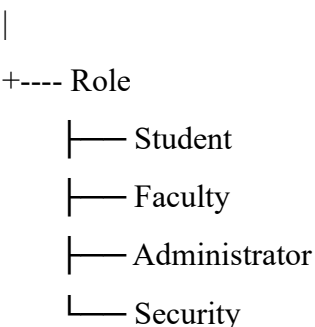
User



However, this approach becomes problematic because the scenario allows one person to have multiple roles, and roles can change over time.

A better alternative is composition using Role objects:

User



Comparison

Criterion	Inheritance	Composition
Multiple roles	Difficult	Easy
Changing roles	Requires changing object type	Roles can be added/removed

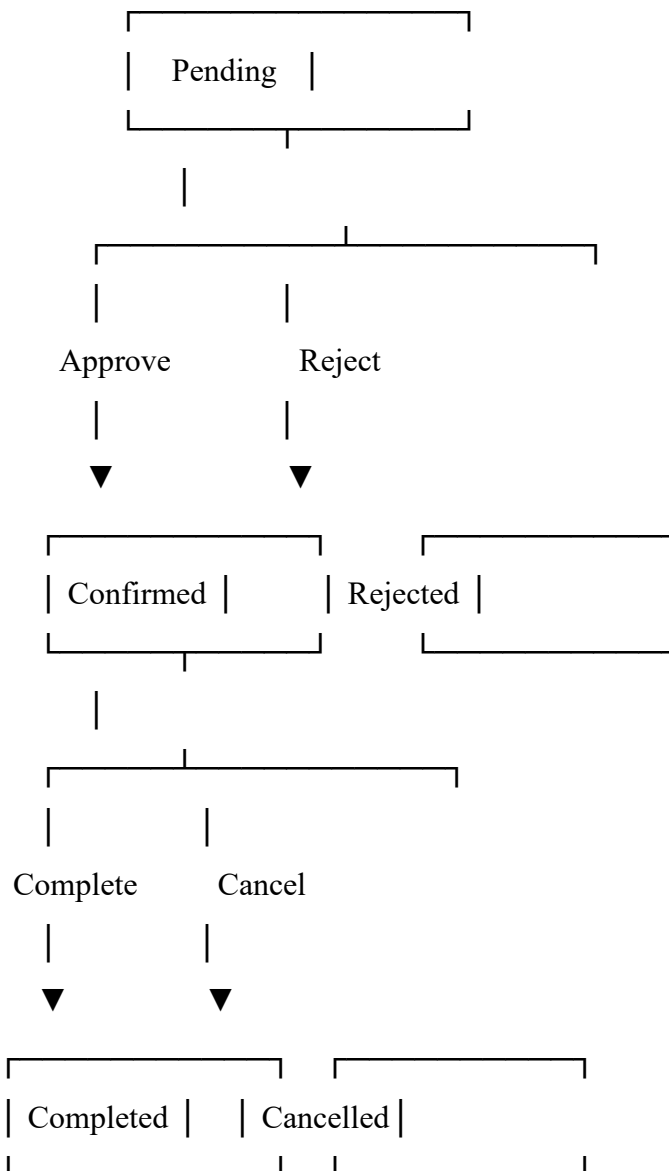
Criterion	Inheritance	Composition
New roles	May require new subclasses	New role objects can be created
Permission management	More tightly coupled	More flexible
Privacy control	Less flexible	Permissions can be assigned to roles

Decision

Composition with Role objects is selected because a person may have multiple roles and role permissions can change. It also supports the Privacy First condition by allowing ordinary users to have limited permissions while authorized security or administration roles receive additional access.

TASK 5

Reservation State Diagram



Confirmed → Relocating → Confirmed

Confirmed → Relocating → Cancelled

Important Decision Cases

1. Simultaneous requests for the same space

The Room checks whether the requested time is available. The Reservation process checks overlapping reservations and applicable priorities. Only a valid request satisfying the rules can become confirmed.

2. Maintenance block after confirmation

If maintenance blocks a room after a reservation has already been confirmed, the Room records the maintenance block. The system identifies affected reservations. The Reservation can then be relocated to a suitable room or cancelled. The affected users receive notifications.

3. Emergency cancellation or relocation

During an emergency, affected confirmed reservations may need to be relocated or cancelled. The Reservation manages its state change, Room checks alternative space availability, and Notification informs affected users.

The assignment specifically requires the system to handle emergency cancellation/relocation and notification of affected users.

TASK 6

Role-Based Access Control

Role-Based Access Control (RBAC) is a useful addition to the Smart Campus Allocation System because the scenario allows users to have multiple roles and allows their permissions to change. In RBAC, permissions are associated with roles rather than being assigned separately to every individual user. NIST describes RBAC as a model in which permitted actions are identified with roles, while users receive permissions through their assigned roles. This makes authorization easier to manage and review. [1]

The proposed model can represent this using the User and Role classes. A user may have roles such as Student, Faculty, Security, or Administrator, and each role can contain specific permissions. For the Privacy First requirement, an ordinary user could have permission to view room availability but not another user's identity or reservation purpose. Authorized security or administration roles could have additional permissions when required for legitimate system activities. Roles can also be changed without changing the basic User object.

Therefore, RBAC improves the model by providing a structured way to enforce privacy and authorization rules while supporting multiple and changing user roles.

References

[1] D. F. Ferraiolo, J. Cugini, and D. R. Kuhn, "Role-Based Access Control (RBAC): Features and Motivations," Proceedings of the 11th Annual Computer Security Applications Conference, pp. 241–248, 1995.

[2] R. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-Based Access Control Models," IEEE Computer, vol. 29, no. 2, pp. 38–47, 1996.